

Compte-Rendu : TD4 - Architecture MVC (modèle, vue, contrôleur)

Matisse KRA
BTS SIO

Compte-Rendu : TD4 - Architecture MVC (modèle, vue, contrôleur)	1
Introduction : Présentation des objectifs du TD	1
Exercice 1 : M - Le modèle	1
Exercice 2 : V - La vue	1
Exercice 3 : C - Le contrôleur	1
Exercice 4 : Le routeur, un autre composant du contrôleur	1
Exercice 5 : Maintenant un vrai routeur	1
Exercice 6 : Vue "détail d'une voiture"	1
Exercice 7 : Vue "ajout d'une voiture"	1
Conclusion	1

Introduction : Présentation des objectifs du TD

Jusqu'ici, le code était écrit de manière monolithique : une même page mélangeait le traitement PHP, les accès à la base de données et les balises HTML. L'objectif de ce TD est de réorganiser le code du TD3 selon le design pattern MVC (Modèle - Vue - Contrôleur), qui sépare ces trois responsabilités. Le modèle gère les données et la persistance, la vue ne contient que le HTML à afficher, et le contrôleur fait l'intermédiaire entre les deux en prenant les décisions. Ce compte-rendu détaille la réorganisation de l'arborescence, la création des vues et du contrôleur, la mise en place d'un routeur qui choisit l'action à exécuter à partir de l'URL, puis l'ajout des actions read, create et created.

Exercice 1 : M - Le modèle

J'ai créé les répertoires config, controller, model, view et view/voiture. J'ai renommé le fichier voiture.php en ModelVoiture.php et la classe Voiture en ModelVoiture, puis j'ai mis la méthode afficher() en commentaire : l'affichage n'est plus le rôle du modèle mais celui de la vue. J'ai aussi corrigé les deux appels à setFetchMode() pour que PDO instancie des objets ModelVoiture et non plus Voiture. J'ai ensuite déplacé ModelVoiture.php et Model.php dans model/, et Conf.php dans config/. Il a fallu corriger le chemin relatif de l'include de Conf.php dans Model.php, qui devient

Matisse
kra

require_once '../config/Conf.php'. Comme le sujet le conseille, j'ai renommé et déplacé les fichiers plutôt que de les copier, pour éviter d'avoir plusieurs versions d'une même classe. Le TD3 restant un rendu à part, j'ai travaillé dans un dossier td4 reprenant les fichiers du TD3.

Index of /bts_sio_2/public_html/PHP/td4

Name	Last modified	Size	Description
 Parent Directory		-	
 TD04_SLAM4.pdf	2026-09-24 14:04	508K	
 config/	2026-09-24 15:24	-	
 controller/	2026-09-24 15:24	-	
 model/	2026-09-24 15:24	-	
 screenshots/	2026-09-24 15:24	-	
 view/	2026-09-24 15:24	-	

Index of /bts_sio_2/public_html/PHP/td4/model

Name	Last modified	Size	Description
 Parent Directory		-	
 Model.php	2026-09-24 15:24	1.7K	
 ModelVoiture.php	2026-09-24 15:24	4.0K	

Exercice 2 : V - La vue

J'ai créé la vue view/voiture/list.php. Ce fichier ne contient que du HTML et une boucle foreach qui parcourt la variable \$tab_v pour afficher l'immatriculation de chaque voiture. La vue n'effectue aucun traitement et ne connaît pas la base de données : elle se contente de lire une variable que le contrôleur a préparée pour elle. Elle n'est d'ailleurs pas appellable directement, puisque \$tab_v n'existerait pas ; son rendu apparaît donc à l'exercice 3, une fois le contrôleur écrit.

Index of /bts_sio_2/public_html/PHP/td4/view/voiture

Name	Last modified	Size	Description
 Parent Directory		-	
 list.php	2026-09-24 15:25	377	

Exercice 3 : C - Le contrôleur

J'ai créé `controller/ControllerVoiture.php`. Il se décompose en trois parties : il charge la déclaration du modèle avec `require_once('../model/ModelVoiture.php')`, il demande au modèle le tableau de toutes les voitures avec `ModelVoiture::getAllVoitures()` qu'il range dans `$tab_v`, puis il appelle la vue avec `require('../view/voiture/list.php')`. Les chemins commencent par `../` car ils sont relatifs au fichier courant, qui se trouve dans `controller/`. J'ai testé la page en appelant l'URL `.../controller/ControllerVoiture.php`. Le résultat affiché est le même qu'avec l'ancien `lireVoiture.php` : c'est bien l'organisation du code qui a changé, pas la page produite. L'ordre d'exécution est le suivant : le contrôleur s'exécute en premier, initialise `$tab_v`, et ce n'est qu'ensuite que la vue est incluse et lit cette variable.

Voiture d'immatriculation AB123CD.

Voiture d'immatriculation CL789VE.

Voiture d'immatriculation GT456KL.

Exercice 4 : Le routeur, un autre composant du contrôleur

Un contrôleur doit gérer plusieurs pages. J'ai donc transformé `ControllerVoiture.php` en une classe, dans laquelle le code précédent devient une méthode statique `readAll()`. On appelle action une telle fonction du contrôleur : une action correspond généralement à une page Web. Il manquait alors le bout de code qui appelle cette action, c'est le rôle du routeur : j'ai créé `controller/routeur.php`, qui charge le

Matisse
kra

contrôleur et appelle `ControllerVoiture::readAll()`. J'ai testé la nouvelle architecture en appelant `.../controller/routeur.php`, qui affiche exactement la même page qu'à l'exercice 3.

Voiture d'immatriculation AB123CD.

Voiture d'immatriculation CL789VE.

Voiture d'immatriculation GT456KL.

Exercice 5 : Maintenant un vrai routeur

Le client doit pouvoir choisir lui-même l'action à exécuter. Il demande donc la page `routeur.php` en transmettant l'action dans l'URL au format query string. J'ai complété `routeur.php` pour qu'il récupère cette information avec `$_GET`, puis appelle la méthode statique correspondante par `ControllerVoiture::$action()`. J'ai ajouté une valeur par défaut (`readAll`) pour que la page reste affichable si aucun paramètre n'est fourni. J'ai testé avec l'URL `.../controller/routeur.php?action=readAll`. Le déroulé complet est donc : le client demande l'URL, le routeur lit `$_GET['action']`, il appelle la méthode `readAll` de `ControllerVoiture`, le contrôleur interroge le modèle, et le contrôleur appelle enfin la vue qui génère la page. À noter que cette écriture appelle une méthode dont le nom vient directement de l'utilisateur, ce qui suppose de contrôler les valeurs acceptées dans une vraie application.

Voiture d'immatriculation AB123CD.

Voiture d'immatriculation CL789VE.

Voiture d'immatriculation GT456KL.

Exercice 6 : Vue “détail d'une voiture”

J'ai créé la vue `view/voiture/detail.php`, qui affiche la marque, la couleur et l'immatriculation de la voiture contenue dans `$v`, comme le faisait l'ancienne méthode `afficher()` aujourd'hui commentée. J'ai ajouté au contrôleur l'action `read` :

Matisse
kra

elle récupère l'immatriculation passée dans l'URL, appelle `ModelVoiture::getVoitureByImmat()`, place le résultat dans `$v` et appelle la vue de détail. J'ai testé avec l'URL `.../routeur.php?action=read&immat=AB123CD`. J'ai ensuite rendu les immatriculations cliquables dans `list.php` : chaque immatriculation est un lien vers `routeur.php?action=read&immat=...`, l'immatriculation étant passée dans `urlencode()` pour rester valide dans une URL. Enfin, comme `getVoitureByImmat()` renvoie `false` quand aucune voiture ne correspond, j'ai créé la vue `view/voiture/error.php` et le contrôleur bascule vers elle dans ce cas, plutôt que de planter en appelant une méthode sur `false`.

Voiture d'immatriculation [AB123CD](#).

Voiture d'immatriculation [CL789VE](#).

Voiture d'immatriculation [GT456KL](#).

[Ajouter une voiture](#)

Marque : peugeot

Couleur : rouge

Immatriculation : AB123CD

[Retour à la liste des voitures](#)

Marque : Renault

Couleur : bleu

Immatriculation : CL789VE

[Retour à la liste des voitures](#)

Erreur

Aucune voiture ne correspond à l'immatriculation ZZ999ZZ.

[Retour à la liste des voitures](#)

Exercice 7 : Vue “ajout d'une voiture”

J'ai créé la vue `view/voiture/create.php` en reprenant le formulaire `formulaireVoiture.html` du TD1, et j'ai ajouté l'action `create` au contrôleur pour l'afficher. La page de traitement du formulaire est l'action `created` du routeur. Comme la méthode du formulaire est `GET`, écrire `action='routeur.php?action=created'` ne fonctionne pas : la query string de l'attribut `action` est écrasée par les données du formulaire. J'ai donc utilisé la solution conseillée, un champ caché `<input type='hidden' name='action' value='created'>`, qui transmet l'action en même temps que les autres champs. J'ai ensuite créé l'action `created`, qui récupère les données dans la query string, instancie un `ModelVoiture`, appelle sa méthode `save()`, puis appelle `readAll()` pour réafficher la liste complète. J'ai testé l'action directement par l'URL avec la voiture `MV111MV`, puis le parcours complet depuis le formulaire avec la voiture `FR222FR` : l'URL obtenue après envoi est bien `routeur.php?action=created&immatriculation=FR222FR&marque=Ford&couleur=gris`, la voiture apparaît dans la liste et j'ai vérifié son enregistrement dans `phpMyAdmin`.

Mon formulaire :

Immatriculation :	FR222FR
Marque :	Ford
Couleur :	gris
Envoyer	

Matisse
kra

Voiture d'immatriculation [AB123CD](#).

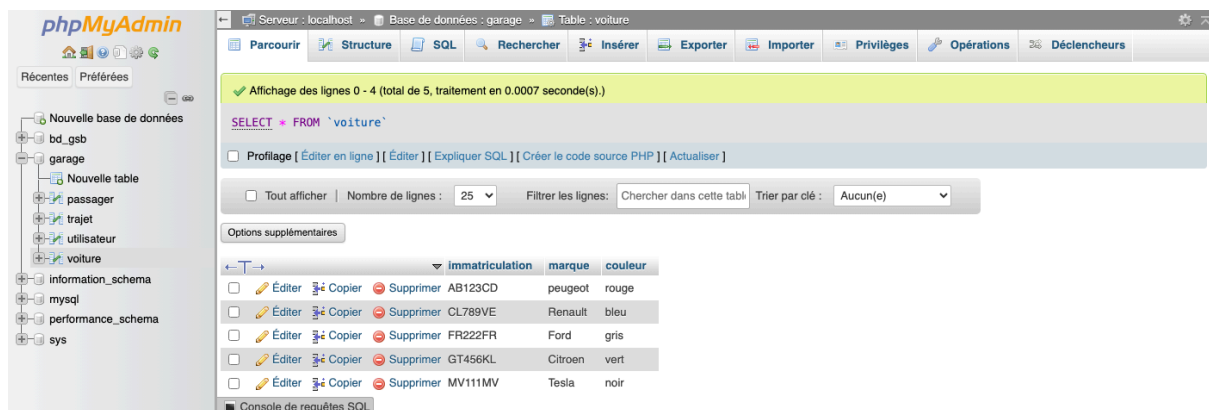
Voiture d'immatriculation [CL789VE](#).

Voiture d'immatriculation [FR222FR](#).

Voiture d'immatriculation [GT456KL](#).

Voiture d'immatriculation [MV111MV](#).

[Ajouter une voiture](#)



Conclusion

Ce TD n'ajoute aucune fonctionnalité par rapport au TD3 : il en réorganise le code. Chaque fichier a maintenant une responsabilité unique, le modèle pour les données, la vue pour le HTML, le contrôleur pour les décisions, et le routeur comme unique point d'entrée du site. C'est ce découpage qui rend l'ajout de fonctionnalités plus simple : une nouvelle page revient désormais à écrire une action dans le contrôleur et une vue, sans toucher au modèle ni aux pages existantes, comme je l'ai constaté en ajoutant successivement read, create et created. Cette architecture prépare aussi la suite, puisque les actions restantes (comme la suppression d'une voiture) viendront se greffer exactement de la même façon.